

FHIR + AI Hackathon

Session 1 Orientation

FHIR Fundamentals with LLM-Assisted Code Generation

Duration: 1 hour

BMI 512 - Clinical Informatics and AI | Stony Brook University

The Big Picture

In this hackathon you will build a clinical data pipeline that turns a plain-English question into an evidence-based answer. The three sessions walk you through this pipeline step by step:

English Question

| (you prompt an LLM)

v

Python / FHIR Code

| (your code queries a FHIR server)

v

Structured Clinical Data (JSON)

| (you prompt an LLM again)

v

Clinical Summary (narrative text)

Key insight: The LLM never touches the FHIR server directly. YOUR CODE does the querying. The LLM helps you write the code and interpret the results.

Three-Session Arc

Session 1 - Manual Pipeline (today)

- You write the queries (with LLM help) and decide the order of steps.
- Goal: understand FHIR resources, references, and multi-step queries.

Session 2 - Observe an Agent

- An AI agent executes the same pipeline autonomously using tool use.
- You predict its behaviour, then compare to reality.

Session 3 - Open Exploration

- You design your own clinical questions with an expanded 6-tool agent.
- Goal: find failure modes and document them. Finding failures = success.

Each session builds on the last. Save your Session 1 notebook -- you will reference it in Sessions 2 and 3.

Today's Clinical Scenario

"Find patients with Type 2 diabetes, retrieve their most recent HbA1c values, and identify those with poor glycemic control (HbA1c > 7.0%)."

This is a real clinical informatics task. To answer it you need to:

- Search a FHIR server for Condition resources (diabetes diagnosis)
- Follow references to retrieve Patient demographics (name, DOB, gender)
- Search for Observation resources (HbA1c lab results) per patient
- Combine and analyse the results to flag patients with poor control
- Generate a narrative clinical summary from the structured data

Note on threshold: In clinical practice 'poor control' varies by guideline (commonly 7.0-9.0%). We use > 7.0% because the Synthea synthetic data on this server skews toward lower HbA1c values. This is a data artifact, not a clinical recommendation.

What is FHIR?

FHIR (Fast Healthcare Interoperability Resources) is an HL7 standard for exchanging healthcare information electronically. Think of it as a RESTful API for clinical data.

Key ideas

- All clinical data is organised into Resources (Patient, Condition, Observation, ...)
- Each resource has a unique ID and a standard JSON structure
- Resources reference each other (e.g. a Condition points to a Patient)
- You query the server with simple HTTP GET requests and URL parameters

The server we are using

- URL: <https://launch.smarthealthit.org/v/r4/fhir>
- Public sandbox -- no login, no API key needed for Session 1
- Populated with synthetic patients generated by Synthea
- FHIR version R4 (the current widely-adopted release)

APIs and JSON in 3 Minutes

What is an API?

An API (Application Programming Interface) lets programs talk to servers. A FHIR server is an API: you send it a URL, it sends back JSON.

```
GET https://launch.smarthealthit.org/v/r4/fhir/Patient?_count=1
```

Response (JSON):

```
{
  "resourceType": "Bundle",
  "total": 1243,
  "entry": [
    {
      "resource": {
        "resourceType": "Patient",
        "id": "abc123",
        "name": [{"given": ["Jane"], "family": "Doe"}],
        "birthDate": "1965-04-12",
        "gender": "female"
      }
    }
  ]
}
```

FHIR Resources You Will Use Today

Condition

Records a diagnosis. Contains the SNOMED CT code, onset date, and a reference to the Patient.

```
GET /Condition?code=44054006&_count=30 --> Bundle of Conditions
```

Patient

Demographics: name, birth date, gender, address. Other resources point here via subject.reference.

```
GET /Patient/{id} --> Single Patient resource
```

Observation

A lab result or vital sign. Contains the LOINC code, numeric value, unit, and date.

```
GET /Observation?subject=Patient/{id}&code=4548-4&_sort=-date&_count=1
```

How FHIR Resources Link Together

Our clinical question requires THREE types of FHIR resources linked by references:

```
Condition (code: 44054006 -- Type 2 Diabetes, SNOMED CT)
  |-- subject.reference ----> Patient/{id}
                               ^
Observation (code: 4548-4 -- HbA1c, LOINC)
  |-- subject -----+
```

To answer our question we must:

- 1. Search Conditions --> find patients with diabetes
- 2. Follow references --> get Patient demographics (name, DOB, gender)
- 3. Search Observations --> get each patient's most recent HbA1c value
- 4. Combine and analyse --> flag patients with HbA1c > 7.0%

This multi-step pattern is fundamental to FHIR: clinical facts are stored in separate, linked resources. No single query gives you the full answer.

Clinical Codes Reference

Code	System	Meaning	Used In	ICD-10
44054006	SNOMED	Type 2 Diabetes Mellitus	Condition search	E11
59621000	SNOMED	Essential Hypertension	(Session 3)	I10
4548-4	LOINC	Hemoglobin A1c (HbA1c)	Observation search	-
85354-9	LOINC	Blood Pressure panel	(Session 3)	-
2160-0	LOINC	Creatinine	(Session 3)	-

HbA1c Interpretation

- < 5.7% Normal
- 5.7 - 6.4% Prediabetes
- >= 6.5% Diabetes
- > 7.0% Poor glycemic control -- needs intervention (our threshold)

IMPORTANT: This FHIR server uses SNOMED CT codes for conditions, NOT ICD-10. Always use SNOMED codes (e.g. 44054006) when searching for conditions.

Your Workflow Today

You will work through the notebook step by step. For each step:

- Read the instruction cell (markdown)
- Ask Claude to generate the Python code
- Paste the code into the empty cell and run it
- Run the verification cell to check your results

The 5 Steps

- Step 0:** Run the setup cell and see your first FHIR query (pre-built)
- Step 1:** Search for Condition resources with Type 2 Diabetes (SNOMED 44054006)
- Step 2:** Follow patient references to get demographics (name, DOB, gender)
- Step 3:** Search for each patient's most recent HbA1c Observation (LOINC 4548-4)
- Step 4:** Generate a clinical summary by prompting Claude with your table

Using Claude as Your Code Generator

In Session 1, you use the Claude web interface (claude.ai) to generate Python code. The notebook tells you exactly what to ask. Here is the workflow:

1. Read the prompt in the notebook (e.g. 'Ask Claude: ...')
2. Copy-paste the prompt into claude.ai
3. Claude generates Python code using the requests library
4. Copy Claude's code into the empty Colab cell
5. Run the cell (Shift+Enter)
6. Run the verification cell to confirm correctness

Good Prompt vs Poor Prompt

GOOD: "Write Python code using requests to search for Condition resources with SNOMED CT code 44054006 on the FHIR server at <https://launch.smarthealthit.org/v/r4/fhir>. Limit to 30 results."

POOR: "Get me diabetes patients from FHIR."

Be specific: include the exact code system, codes, server URL, and desired output format.

Understanding FHIR Bundles

Every FHIR search returns a Bundle -- a container that wraps the results:

```
{
  "resourceType": "Bundle",
  "total": 30,                <-- total matches on server
  "entry": [                 <-- array of results
    {
      "resource": {           <-- the actual clinical resource
        "resourceType": "Condition",
        "id": "cond-001",
        "subject": {
          "reference": "Patient/abc123"  <-- LINK to a Patient
        },
        "code": {
          "coding": [{"code": "44054006", ...}]
        }
      },
    },
    ...                       <-- more entries
  ]
}
```

- bundle['total'] tells you how many matched on the entire server
- bundle['entry'] is the list of results (may be paginated)
- entry['resource'] is the actual FHIR resource (Condition, Observation, etc.)

Google Colab Quick Reference

The notebooks run in Google Colab -- a free cloud Jupyter environment.

Key actions

- Shift+Enter -- run the current cell and advance to the next
- Cells run in order. Variables persist between cells.
- Code cells (gray background) contain Python. Markdown cells contain instructions.
- If you see a play button on the left of a cell, click it to run.

Colab Secrets (for Session 2 and 3 only)

- Click the key icon in the left sidebar to open Secrets
- Store your ANTHROPIC_API_KEY there -- not in your code!
- Session 1 does NOT need any API keys -- just the FHIR server

If you lose your work: Colab autosaves to Google Drive. You can also File > Download .ipynb to save a local copy.

Key Learning Points

1. LLM as Code Generator

You describe what you want in plain English, Claude writes the Python. You inspect the code, paste it in, and run it. The LLM is your assistant, not the pilot.

2. Multi-Step Clinical Queries

No single FHIR query answers our clinical question. You must chain Condition -> Patient -> Observation queries and combine the results. This is how real clinical informatics works.

3. Separation of Concerns

The LLM generates code and interprets results. YOUR CODE queries the FHIR server. This separation is critical for safety: the LLM never has direct database access. In Session 2, the LLM will request tool calls, but your code still executes them.

What to Expect

Success looks like:

- ~30 patients with Type 2 Diabetes found (from Condition search)
- Demographics retrieved for all patients
- HbA1c values for most patients (some may have no lab data -- that's normal)
- A few patients flagged with poor control (HbA1c > 7.0%)
- A clinical summary paragraph generated by Claude

Common issues:

- Claude's code uses a different variable name than expected -- the verification cell will tell you what to rename
- Using ICD-10 codes instead of SNOMED CT -- use 44054006, not E11
- FHIR server returns 500/502 errors -- ask the instructor for the backup notebook
- Some patients have no HbA1c data -- expected, the verification cell handles it

Next Session Preview

Session 2: Tool Use & AI Agents

Today you are the orchestrator:

- You decide which query to run next
- You paste code, run it, inspect results, decide the next step
- The LLM only helps write code when you ask

In Session 2 the LLM takes over orchestration:

- You give the LLM a question and a set of tools (functions it can call)
- The LLM decides which tool to call, with what arguments, and in what order
- Your code executes the tool calls and feeds results back to the LLM
- The LLM synthesises a final answer from all the data it gathered

This is called "tool use" or "function calling." The manual steps you do today become the tools the agent uses autonomously in Session 2.

Questions?

Before you start:

- Open the Session 1 notebook in Google Colab
- Run the first (Setup) cell to verify the FHIR server is reachable
- Have the Claude web interface (claude.ai) open in another tab
- Keep this orientation handy for the clinical codes reference

Let's begin!

Resources

FHIR Server

<https://launch.smarthealthit.org/v/r4/fhir>

Claude Web Interface

<https://claude.ai> (free tier is sufficient for Session 1)

FHIR Documentation

<https://hl7.org/fhir/R4/>

Key Codes to Remember

- Type 2 Diabetes (SNOMED CT): 44054006
- HbA1c (LOINC): 4548-4
- Poor glycemic control threshold: > 7.0%

REMEMBER: Use SNOMED CT codes (not ICD-10) for all condition searches on this server! The most common mistake is searching with ICD-10 code E11 instead of SNOMED 44054006.